

Виртуальное блочное устройство на базе оперативной памяти в ядре Linux

Климович Станислав Андреевич

Группа: 25.Б41-мм

Кафедра: системного программирования

Научный руководитель: Васенина Анна Игоревна

Номер семестра практики: 2

1. Введение

В Unix-подобных операционных системах всё является файлом: блочное устройство представляет собой накопитель данных или его раздел, доступный через виртуальную файловую систему (VFS) и пригодный для монтирования с файловой системой поверх. Такая абстракция позволяет единообразно работать с реальным оборудованием и программными (виртуальными) устройствами. Виртуальные блочные устройства применяются в облачных средах, системах виртуализации и встроенных системах — там, где реального дискового оборудования нет или оно избыточно. Понимание их внутреннего устройства необходимо разработчикам storage-стека, гипервизоров и авторам вкладов в ядро Linux.

В рамках данной учебной практики реализован модуль ядра `/dev/ram_virtblk` — простейший RAM-блочник (базовый уровень сложности в иерархии виртуальных блочных устройств курса) — с CI/CD-инфраструктурой и автоматизированным интеграционным тестированием на реальном ядре. Полученные навыки применимы при разработке драйверов, storage-подсистем и участии в open-source проектах Linux-экосистемы.

2. Постановка задачи

Целью работы является разработка модуля ядра Linux, реализующего виртуальное блочное устройство `/dev/ram_virtblk`, поддерживающее форматирование, монтирование и файловые операции с хранением данных в RAM.

Для достижения цели требуется решить следующие задачи:

1. Изучить подсистему блочных устройств ядра Linux и существующие реализации RAM-дисков.
2. Реализовать модуль ядра на основе BIO-based API с настроенной CI/CD-инфраструктурой: линтингом и автоматической генерацией документации.
3. Верифицировать корректность устройства интеграционными тестами на целевом ядре (6.18.13-200.fc43.x86_64).

3. Обзор

Цель обзора — выбрать архитектурный подход для реализации и оценить применимость существующих решений в качестве основы.

В ядре Linux существуют штатные RAM-диски: `brd` хранит данные в `radix-tree` из страниц памяти и поддерживает разделы; `zram` дополнительно сжимает данные алгоритмами `lz4/zstd`; `loop`-устройство монтирует файлы образов как блочные устройства. Все используют многоочередной стек `blk-mq` и представляют собой производственный код объёмом от нескольких сотен до тысяч строк.

Архитектура блочного устройства в ядре предполагает два подхода к обработке I/O: классический на основе очереди запросов (`blk-mq`) и упрощённый BIO-based — прямой обработчик `submit_bio` без промежуточной диспетчеризации. Для RAM-устройства BIO-based API не создаёт потерь производительности, так как отсутствует аппаратная латентность, ради которой `blk-mq` и разрабатывалось.

Вывод: `brd` и `zram` избыточны для учебной задачи и скрывают именно те механизмы, которые необходимо освоить. BIO-based API выбран как оптимальный: минимальный по сложности и достаточный по функциональности.

4. Описание предлагаемого решения

Структура модуля. Три файла: `virtblk.h` — константы устройства (имя, размер сектора, количество секторов, суммарный объём 64 МиБ); `virtblk.c` — реализация; `Makefile` — сборка через стандартный механизм `Kbuild`.

Управление памятью. RAM-буфер (131072 сектора × 512 байт) выделяется через `vzalloc()`. От `kmalloc()` отказались: 64 МиБ не требуют физически непрерывной памяти; `vzalloc()` работает с виртуально непрерывными, но физически разрозненными страницами, снижая давление на аллокатор ядра.

Поток I/O. Ядро вызывает `ram_virtblk_submit_bio()` с BIO-запросом. Функция вычисляет байтовое смещение (`bi_sector × 512`), проверяет выход за границы (при превышении — `bio_io_error()`), затем обходит сегменты через `bio_for_each_segment()`. Для каждого сегмента `ram_virtblk_transfer()` маппит физическую страницу через `kmap_local_page()` (thread-safe замена устаревшего `kmap()`), выполняет `memcpy()` в/из буфера и немедленно размаппирует через `kunmap_local()`, предотвращая утечки маппингов.

Инициализация и выгрузка реализованы через цепочку `goto`: `vzalloc()` → `register_blkdev()` → `blk_alloc_disk()` → `add_disk()`. При ошибке на шаге *N* все предыдущие шаги откатываются в обратном порядке. Флаг `GENHD_FL_NO_PART` запрещает сканирование таблицы разделов — устройство монолитно, без разделов.

CI/CD. Два воркфлоу GitHub Actions. `lint.yml`: `checkpatch.pl` — официальный линтер ядра Linux — проверяет стиль кода; `Doxxygen` генерирует документацию из исходных комментариев и публикует на GitHub Pages. `test.yml`: сначала проверяется точная версия ядра на `guppper-e`, затем собирается и запускается `.ko`.

5. Тестирование

Модульное тестирование кода ядра в рамках данной практики не применяется: модуль исполняется в `kernel space` и не запускается в изоляции; специализированные средства (например, KUnit) остаются за пределами задачи. Инструмент KASAN (аналог ASAN для ядра) требует пересборки с `CONFIG_KASAN=y`, что также выходит за рамки задания. Вместо этого выполнены четыре интеграционных теста на реальном устройстве (Fedora Server 43, ядро 6.18.13-200.fc43.x86_64, гипервизор VirtualBox):

1. **Базовый write/read** — запись строки через `dd` в сырое устройство, чтение и побайтовое сравнение.
2. **Файловая система** — форматирование в `ext4` (`mkfs.ext4`), монтирование, запись и чтение файла через VFS.
3. **Целостность данных** — запись блока случайных данных из `/dev/urandom`, побайтовое сравнение снимка через `cmp`.
4. **Граница устройства** — I/O-запрос за конец устройства; ожидается вызов `bio_io_error()` и ошибка на стороне ядра.

Тесты запускаются автоматически при каждом пуше через GitHub Actions и проходят успешно: устройство корректно выполняет I/O, монтируется как `ext4`, сохраняет побайтовую целостность данных и возвращает ошибку при выходе за границы.

6. Заключение

В ходе выполнения работы получены следующие результаты:

- Проведён обзор существующих реализаций RAM-дисков ядра Linux (`brd`, `zram`, `loop`) и выбран BIO-based подход как оптимальный для учебной задачи.
- Реализован модуль ядра Linux (`virtblk.c`, ≈ 150 строк) на основе BIO-based API: регистрация блочного устройства, обработчик `submit_bio`, инициализация и выгрузка с обработкой ошибок.
- Зарегистрировано виртуальное блочное устройство `/dev/ram_virtblk` ёмкостью 64 МиБ, поддерживающее форматирование в `ext4`, монтирование и стандартные файловые операции.
- Настроен CI/CD-пайплайн: линтинг `checkpatch.pl` и автогенерация документации Doxygen на GitHub-hosted runner; интеграционные тесты на self-hosted runner с целевой версией ядра.
- Написаны и автоматизированы четыре интеграционных теста, покрывающих базовый I/O, файловую систему, целостность данных и граничные случаи.

Репозиторий: <https://github.com/Monrealle/linux-virtblk>

Техническая документация: <https://monrealle.github.io/linux-virtblk/>

Список литературы

- [1] Ловягин Н. Ю., Ловягин Ю. Н. Основы работы в Unix-подобных операционных системах: учебное пособие. Ч. I. — СПб., 2020. — 157 с. — (§1.8 Файловая система; §E.7 Монтирование разделов; §E.8 Работа с разделами и файловыми системами). — URL: <https://api.dspace.spbu.ru/server/api/core/bitstreams/b95862a3-b6a5-45b8-9a68-283a7291dbbf/content>.
- [2] Corbet J., Rubini A., Kroah-Hartman G. Linux Device Drivers [Электронный ресурс]. — 3-е изд. — O'Reilly Media, 2005. — 636 с. — (Главы 1, 2, 4, 16). — URL: <https://lwn.net/Kernel/LDD3/>.
- [3] Linux kernel coding style // The Linux Kernel documentation. — URL: <https://docs.kernel.org/process/coding-style.html>